
Assignment 1

46115 Turbulence Modeling

Authors


Szczepan Letkiewicz - s244067

November 18, 2024

Contents

1	Introduction	1
2	Governing Equations	1
3	Modelling Turbulence	2
3.1	Prandtl's Algebraic model	3
3.2	k model	4
4	Model Analysis & Comparison	6
4.1	Results	6
4.2	Effect of van Driest damping	9
4.3	Comparison of Models	9
5	Comparison with Laminar Flow	9
6	Conclusion	10
	List of Figures	I
	List of Tables	I
	References	I
7	Appendix	II
7.1	Acknowledgments	II
7.2	Contributions	II
7.3	Code	II

1 Introduction

This report focuses on the modelling of a turbulent fully developed channel flow using Reynolds-Averaged Navier Stokes (RANS) equations. To address the turbulence closure problem, the algebraic Prandtl and k-epsilon models are implemented, allowing for a comparative analysis with DNS data from Moser et al. [1]. Moreover, the importance of adding a van Driest damping function in modelling wall-bounded flows is investigated. The structure of the report is summarized as follows. Firstly, the governing RANS equations and relevant flow parameters for Poiseuille flow are described in section 2. This is then followed by section 3, which represents the derivations and implementation of the two turbulence models. Next, the results from the models are presented in section 4 and a comparison of the turbulent velocity profile with the laminar equivalent is made in section 5. Concluding thoughts and outcomes drawn from the investigation are outlined in section 6.

2 Governing Equations

The turbulent channel flow case is described using the RANS momentum equation in the streamwise direction. It links the streamwise pressure gradient as a function of the channel geometry and flow parameters. Consider the general form of (Equation 1).

$$\frac{D u_i}{Dt} = -\frac{1}{\rho} \frac{\partial P}{\partial x} + \frac{\partial}{\partial x} \left(\frac{\tau_{ix}}{\rho} \right) + \frac{\partial}{\partial x} \left(\frac{\tau_{ix}^0}{\rho} \right) - \overline{u_i u_j} \frac{\partial u_j}{\partial x} \quad (1)$$

The flow is assumed to be composed of a Newtonian, incompressible fluid which is one-dimensional and fully developed. These conditions yield a more simplified equation to describe the flow, as described by Equation 2. This equation can then be simplified further to express the pressure gradient $\frac{\partial P}{\partial x}$, in terms of the half-channel height, H , the fluid density, ρ , and the friction velocity, u^* , as shown in Equation 3.

$$0 = -\frac{1}{\rho} \frac{\partial P}{\partial x} + \frac{\partial \tau}{\partial y} - \frac{\partial \tau^0}{\partial y} \quad (2)$$

Isolating the pressure term by rearranging the equation, using $\tau = \mu \frac{\partial u}{\partial y}$, yields the following:

$$\frac{\partial P}{\partial x} = \frac{\partial \tau}{\partial y} - \frac{\partial \tau^0}{\partial y}$$

Integrating both sides for the half-channel height gives the following:

$$\int_0^H \frac{\partial P}{\partial x} dy = \int_0^H \left(\frac{\partial \tau}{\partial y} - \frac{\partial \tau^0}{\partial y} \right) dy$$

$$\frac{\partial P}{\partial x} H = \left[\tau \right]_{y=0}^{y=H} - \left[\tau^0 \right]_{y=0}^{y=H}$$

$\frac{\partial P}{\partial x} H = \tau|_{y=H} - \tau|_{y=0} - \tau^0|_{y=H} + \tau^0|_{y=0}$

$\frac{\partial P}{\partial x} H = \tau|_{y=H} - \tau|_{y=0} - \tau^0|_{y=H} + \tau^0|_{y=0}$

In the resulting expression, many terms are equal to zero due to the listed reasons:

- The velocity profile of the channel is symmetric, meaning that at the half-channel height, H , the velocity is constant, and thus its derivative, $\frac{\partial u}{\partial y}$, is zero.
- For the flow in question, the Prandtl approximation $\overline{u'v'} = -\frac{2}{3} \frac{\partial \overline{u}}{\partial y} \frac{u^2}{H}$ can be applied. Since $\frac{\partial u}{\partial y} = 0$, this term is also zero.
- At the wall, there is a no-slip condition, meaning that the velocity is zero, and thus any velocity fluctuations are also zero.

Now, it is possible to substitute in the friction velocity, u_w , which yields the following expression:

$$\frac{\partial \overline{P}}{\partial x} = \frac{1}{H} \frac{\partial \overline{u}}{\partial y} \bigg|_{y=0} = \frac{u_w}{H} = \frac{u^2}{H}$$

Alternatively, this can also be re-written in a different form, involving friction Reynolds number Re_w , for the ease of implementation into the models later on:

$$\frac{\partial \overline{P}}{\partial x} = \frac{u^2}{H} = \frac{Re_w^2}{H^3}; \text{ where } Re_w = \frac{u_w H}{\nu} \quad (3)$$

Consequently, Equation 3 describes the streamwise pressure gradient for the turbulent channel flow in question. If a laminar flow was considered instead, a derivation similar to this one could be carried out which would yield the same result. This is the case because the turbulent components of the momentum equation cancel out during the derivation, as shown above, leading to an equivalent final result for both laminar and turbulent flows (Equation 4). However, one should be aware that frictional velocity has no physical meaning in terms of laminar flow as it is a parameter strictly related to turbulence. Therefore, the derivation for the laminar case is carried out purely for the sake of comparison with the turbulent case while being aware of its fundamental faultiness.

$$\frac{\partial \overline{P}}{\partial x} \bigg|_{\text{turbulent}} = \frac{\partial \overline{P}}{\partial x} \bigg|_{\text{laminar}} = \frac{u^2}{H} \quad (4)$$

3 Modelling Turbulence

The present flow case is described using the governing Equation 2, in terms of two unknown parameters; mean velocity $\overline{u}$ and Reynolds stress term $\overline{u'v'}$. This means a turbulence closure problem exists and a model is required to solve the flow case.

This section models the flow using two methods based on linear Eddy Viscosity. These models are implemented using the Boussinesq hypothesis given in Equation 5, which predicts that turbulent flow is dominated by mixing due to large energetic turbulent eddies.

$$\overline{u_i'v_j'} = -2 \nu_t \left(\frac{\partial \overline{u_i}}{\partial x_j} + \frac{\partial \overline{u_j}}{\partial x_i} \right) + \frac{2}{3} \delta_{ij} k \quad ! \quad \overline{u'v'} = -\nu_t \frac{\partial \overline{u}}{\partial y} \quad (5)$$

$\underbrace{\left(\frac{\partial \overline{u_i}}{\partial x_j} + \frac{\partial \overline{u_j}}{\partial x_i} \right)}_{\text{Strain Tensor } S_{ij}}$

Applying this theory to the flow case in question yields the following:

$$-\frac{1}{\rho} \frac{\partial \bar{P}}{\partial x} (H - y) + \nu \left(\frac{\partial \bar{u}}{\partial y} \right) = -\nu_t \left(\frac{\partial \bar{u}}{\partial y} \right) \quad (6)$$

The new term ν_t is known as the eddy viscosity, which represents no physical characteristic of the fluid. Instead, it is a theoretical function of the local flow conditions [2]. Eddy viscosity models aim to quantify this ν_t term in a variety of methods; two of these are explored in this report, namely Prandtl's Algebraic model and the $k - \epsilon$ model.

3.1 Prandtl's Algebraic model

Prandtl's mixing length hypothesis, given in Equation 7, states that $\nu_t \propto u_t l_t$. Here, the ν_t is the product of a velocity scale u_t and length scale l_t , since $\nu_t [\text{m}^2/\text{s}] = [\text{m}] \cdot [\text{m}/\text{s}]$.

$$\nu_t = u_t l_t \quad (7)$$

Furthermore, Prandtl's second hypothesis, given in Equation 8, predicts the velocity scale u_t based on an order of magnitude argument.

$$u_t \approx l_t \left| \frac{\partial \bar{u}}{\partial y} \right| \quad (8)$$

This empirical model can now be applied to Equation 6 to close the turbulence problem and solve the flow case. To do so, consider how the ordinary differential equation is rearranged in terms of the velocity gradient $\frac{\partial \bar{u}}{\partial y}$.

$$l_t \left(\frac{\partial \bar{u}}{\partial y} \right)^2 + \nu \left(\frac{\partial \bar{u}}{\partial y} \right) - \frac{1}{\rho} \frac{\partial \bar{P}}{\partial x} (H - y) = 0$$

$$\frac{\partial \bar{u}}{\partial y} = \frac{-\nu \pm \sqrt{\nu^2 + 4 \frac{1}{\rho} \frac{\partial \bar{P}}{\partial x} (H - y) \cdot l_t^2}}{2l_t^2}$$

Now, this problem can be solved by discretizing the velocity gradient using a forward scheme and arranging the discretized equations in a matrix system of equations: $\mathbf{A} \bar{\mathbf{u}} = \mathbf{B}$.

$$\frac{u(y + dy) - u(y)}{dy} = \underbrace{\frac{-\nu \pm \sqrt{\nu^2 + 4 \frac{1}{\rho} \frac{\partial \bar{P}}{\partial x} (H - y) \cdot l_t^2}}{2l_t^2}}_B \quad (9)$$

Considering the flow case in question, the Dirichlet boundary condition is applied at $y = 0$ due to the no-slip condition, and the Neumann boundary condition is applied at $y = H$ due to the symmetry of the channel. Furthermore, the $\pm$ sign is chosen as positive since the

velocity increases with distance y , meaning that a positive gradient is present. Consider how the matrices should look.

$$\frac{1}{dy} \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ -1 & 1 & 0 & \dots & 0 \\ 0 & -1 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} \bar{u}_1 \\ \bar{u}_2 \\ \vdots \\ \vdots \\ \bar{u}_N \end{pmatrix} = \begin{pmatrix} 0 \\ B_2 \\ \vdots \\ B_{N-1} \\ 0 \end{pmatrix} \quad (10)$$

Before solving the system of equations for velocity ($\bar{u} = \mathbf{A}^{-1}\mathbf{B}$), the pressure term is expressed using Equation 3 and the mixing length scale l_t is replaced by a function. The first function considered is for undamped wall-bounded flows, given by $l_t = \kappa y$ where $\kappa = 0.41$, and secondly, the van Driest damping function given by equation 11, where $A^+ = 26$.

$$l_t = \kappa y \left(1 - e^{\frac{-y^+}{A^+}} \right) \quad (11)$$

The distance y present in both length scales is modelled using a mesh that is spaced apart by Δy . A spacing of 0.001 was selected such that the following condition is satisfied for the large $Re_\tau = 590$.

$$\Delta y = \frac{y^+ \nu}{u_\tau} = \frac{H}{Re_\tau}$$

3.2 $k - \epsilon$ model

To model Reynolds stress with higher fidelity than by using Prandtl's model, the $k - \epsilon$ model is introduced. Unlike Prandtl's model, which is also known as the zero-equation model because it does not introduce any new equations, the $k - \epsilon$ model is a two-equation model that implements two additional equations. It models eddy viscosity as a function of turbulent kinetic energy k and its dissipation rate ϵ according to the following equation:

$$\nu_t = C_\mu f_\mu \frac{k^2}{\epsilon}, \quad (12)$$

where $C_\mu = 0.09$ and f_μ represent the damping function, which captures the effect of viscosity in the viscous sublayer in dampening the eddies and their size reduction, given in Equation 18.

The two equations for k and ϵ are transport equations, which describe how the quantities vary in space throughout the flow. They are formulated as follows:

$$\frac{Dk}{Dt} = P_k - \epsilon + \frac{\partial}{\partial x_j} \left[\left(\nu + \frac{\nu_t}{\sigma_k} \right) \frac{\partial k}{\partial x_j} \right], \quad (13)$$

$$\frac{D\epsilon}{Dt} = C_{\epsilon 1} P_k \frac{\epsilon}{k} - C_{\epsilon 2} f_\epsilon \frac{\epsilon^2}{k} + \frac{\partial}{\partial x_j} \left[\left(\nu + \frac{\nu_t}{\sigma_\epsilon} \right) \frac{\partial \epsilon}{\partial x_j} \right], \quad (14)$$

where $C_{\epsilon 1} = 1.44$, $C_{\epsilon 2} = 1.92$, P_k stands for turbulence production term, defined in Equation 15, and f_ϵ is another damping function accounting for low Reynolds number and wall proximity effects, given in Equation 19.

$$P_k = \nu \left(\frac{\partial \bar{u}}{\partial y} \right)^2 \quad (15)$$

The transport equations for k (Equation 16) and ϵ (Equation 17) can be simplified for the considered channel flow. As the flow is one-directional and fully developed, the substantial derivative of any flow property is zero ($\frac{D}{Dt} = 0$), meaning that the transport equations can be simplified to the following forms:

$$0 = P_k - \epsilon + \frac{\partial}{\partial y} \left[\left(\nu + \frac{\nu_t}{\sigma_k} \right) \frac{\partial k}{\partial y} \right], \quad (16)$$

$$0 = C_{\epsilon 1} P_k \frac{\epsilon}{k} - C_{\epsilon 2} f_\epsilon \frac{\epsilon^2}{k} + \frac{\partial}{\partial y} \left[\left(\nu + \frac{\nu_t}{\sigma_\epsilon} \right) \frac{\partial \epsilon}{\partial y} \right]. \quad (17)$$

The previously mentioned damping functions are removed for the undamped model and described by van Driest damping equations below for the damped model.

$$f_\mu = \left[1 - e^{\left(-\frac{y^+}{26} \right)} \right]^2 \quad (18)$$

$$f_\epsilon = \left[1 - \frac{2}{9} e^{\left(-\left(\frac{Re_\tau}{6} \right)^2 \right)} \right] \left[1 - e^{\left(-\frac{y^+}{5} \right)} \right]^2 \quad (19)$$

To implement this model, these equations are discretized using finite differencing and solved iteratively in combination with the RANS momentum equations until model convergence is achieved. This iteration process begins by providing initial estimates for the values of k and ϵ used to compute the eddy viscosity. Using the same finite differencing method of solving a system of linear equations as in Prandtl, the transport equation for ϵ is solved first, followed by the equation for k and the RANS momentum equation for $\bar{u}$. The eddy viscosity is also updated after each computation.

All three systems of linear equations also contain boundary conditions where, the Dirichlet boundary condition is enforced at the wall for all flow variables apart from ϵ , whose value is determined by Myong and Kasagi [3], whilst in the middle of the channel, the Neumann boundary condition is applied to all flow variables. These boundary conditions are summarized in Table 1.

Table 1: Boundary conditions for k-epsilon model for the fully developed channel flow.

	$\bar{u}$	k	ϵ
At the wall ($y = 0$)	$\bar{u} = 0$	$k = 0$	$\epsilon = \nu \frac{\partial^2 k}{\partial y^2}$
Middle of the channel ($y = H$)	$\frac{\partial \bar{u}}{\partial y} = 0$	$\frac{\partial k}{\partial y} = 0$	$\frac{\partial \epsilon}{\partial y} = 0$

Convergence of an explicit numerical method is important to ensure that the solution approaches a stable and physically realistic result. For this reason, tolerance criteria have been set when implementing the model.

4 Model Analysis & Comparison

4.1 Results

Figure 1 plots the velocity profiles in wall coordinates (y^+ vs. U^+). This is done for both the Prandtl (left column) and $k-\epsilon$ (right column) models, and are compared directly with the corresponding DNS (Direct Numerical Simulation) data [1]. For both models, the results are presented with and without applying the van Driest damping function, as was presented by Equation 18. Furthermore, the results have been presented across three friction Reynolds numbers, as defined in Equation 3,: top row - $Re_\tau = 180$, middle row - $Re_\tau = 395$, and bottom row - $Re_\tau = 590$.

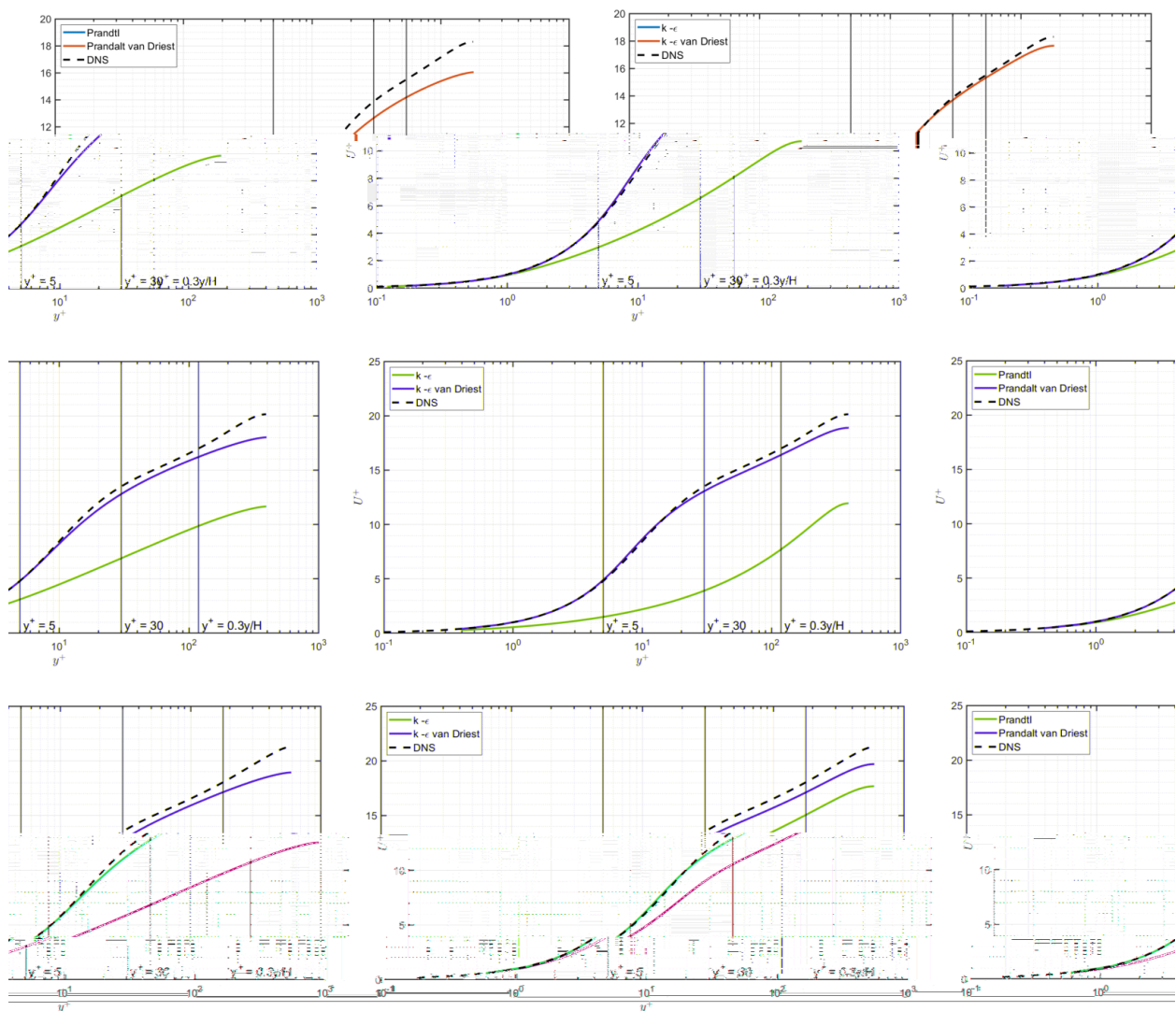


Figure 1: Mean velocity profile: — Damped Prandtl/ $k-\epsilon$, — Undamped Prandtl/ $k-\epsilon$, — DNS; for $Re_\tau = 180$ (Row 1), $Re_\tau = 395$ (Row 2), $Re_\tau = 590$ (Row 3).

The mean velocity profile is plotted in a semi-logarithmic format, primarily for comparative reasons. For instance, the DNS data has been collected from an experiment which likely used different physical conditions such as; temperature, pressure, velocity, etc... However, in a dimensionless format described by the Reynolds number, they experience similar physical behaviour and can be compared. This also makes it easier to visualize it across the distinct flow regions, namely: the viscous sublayer, buffer layer, and logarithmic region. In Figure 1, these regions are indicated by specific y^+ values found in literature [2]. One indicator of the model's accuracy is its correlation with DNS in the viscous sublayer; which is described analytically by $Y^+ = U^+$ and therefore should present a near-perfect match. Secondly, the logarithmic region, described by the logarithmic law [4], should resemble a linear relation in the semi-logarithmic format. The buffer region, on the other hand, lacks a mathematical description and is therefore only compared to the DNS.

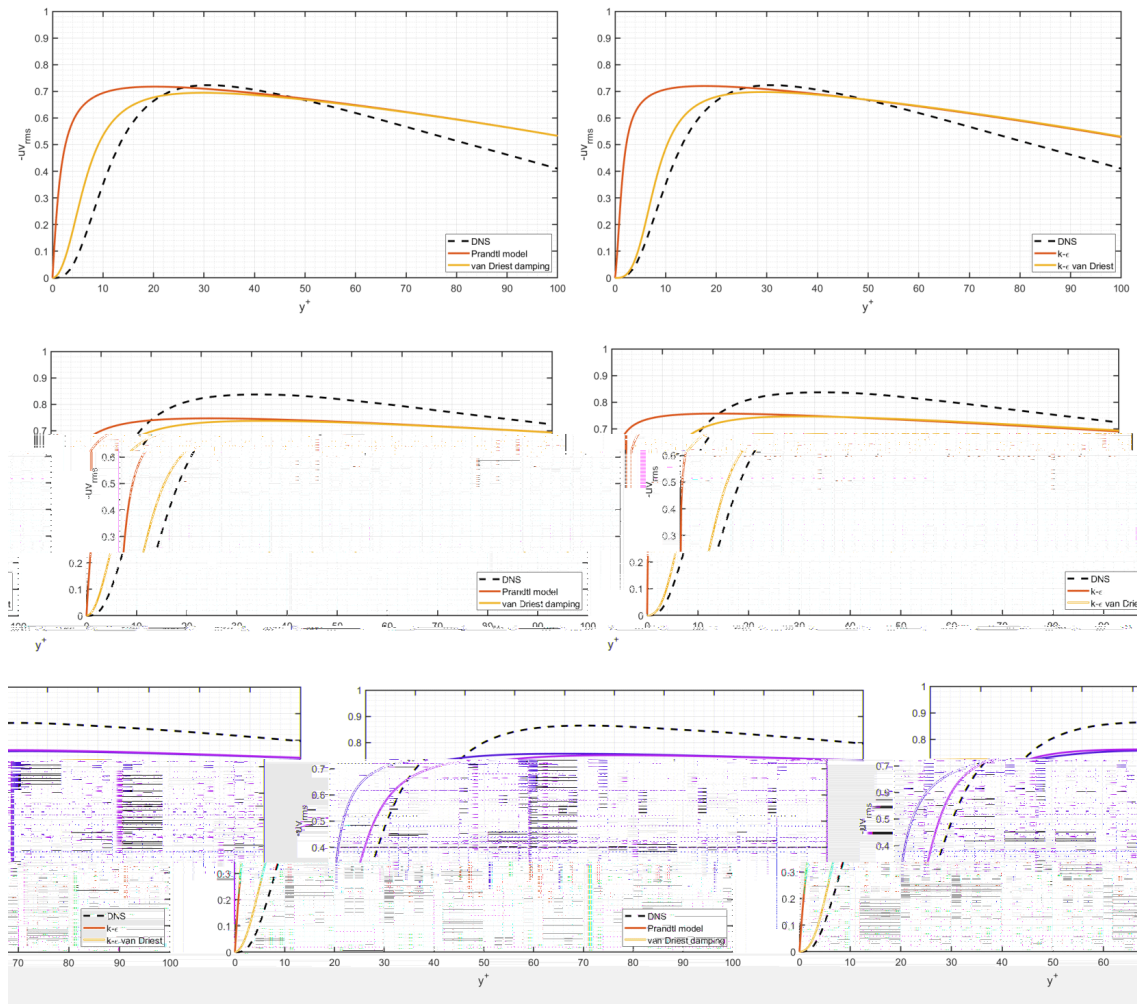


Figure 2: Reynolds shear stress $-\overline{u'v'}$ normalized by the wall shear velocity in wall coordinates — Damped Prandtl/ $k - \epsilon$, — Undamped Prandtl/ $k - \epsilon$, — DNS; for $Re_\tau = 180$ (Row 1), $Re_\tau = 395$ (Row 2), $Re_\tau = 590$ (Row 3).

The plots in Figure 1 also show that as Re_τ increases, the logarithmic region starts further from the wall. This shift is due to the higher friction velocity at larger Re_τ caused by increased wall shear stress, delaying the start of the logarithmic region. Furthermore, the plots using a semi-logarithmic format also facilitate the evaluation of how well the Prandtl and $k - \epsilon$ models capture the velocity profile, especially near the wall where viscous effects are strongest.

In addition to the velocity profile, it is also possible to plot the relevant Reynolds shear stresses acting in the uv direction. These stresses essentially describe the turbulent behaviour of the channel flow and can be plotted as a function of y^+ (Figure 2) or y/H (Figure 3) as stated by the provided literature [5].

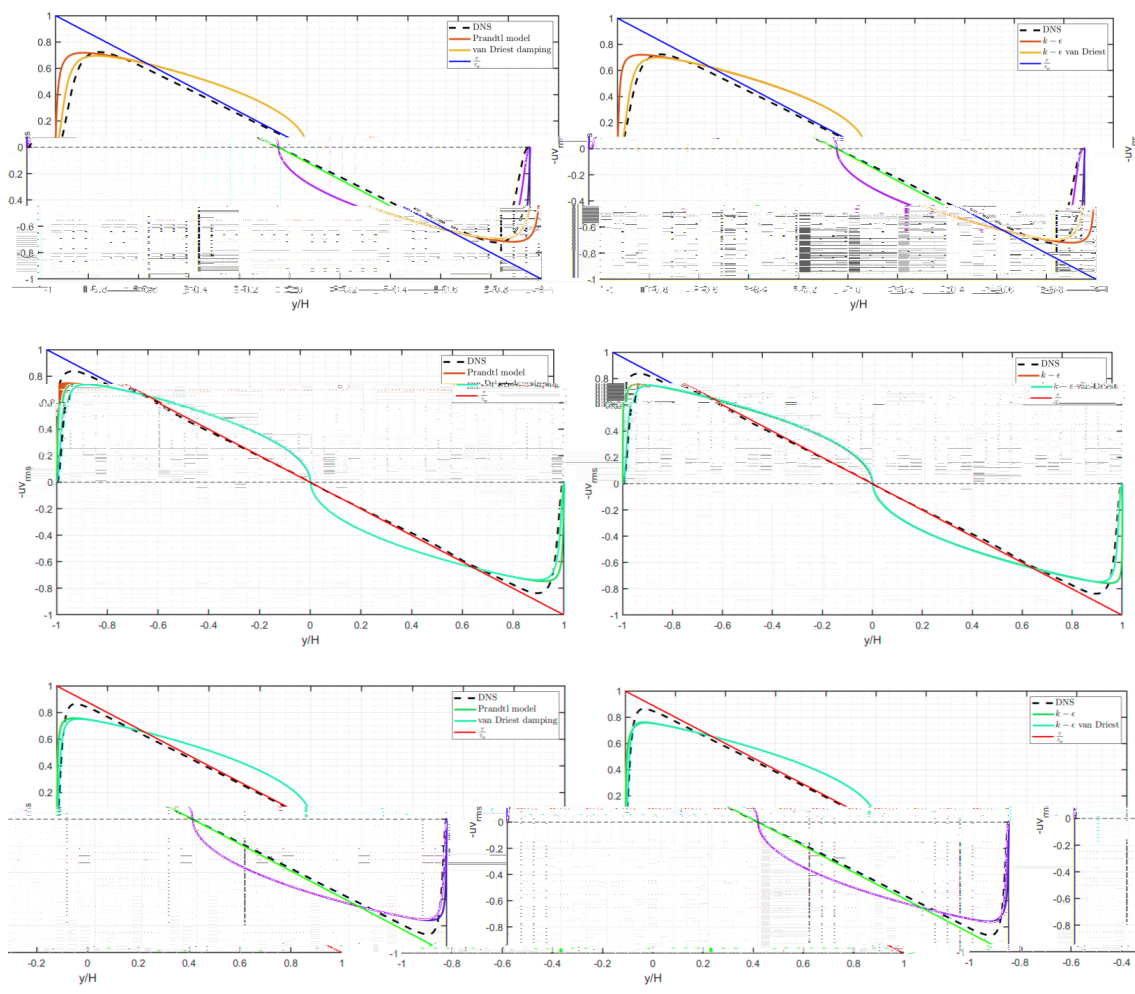


Figure 3: Reynolds shear stress $-\overline{u'v'}$ normalized by the wall shear velocity in global coordinates — Damped Prandtl/ $k - \epsilon$, — Undamped Prandtl/ $k - \epsilon$, — DNS; for $Re_\tau = 180$ (Row 1), $Re_\tau = 395$ (Row 2), $Re_\tau = 590$ (Row 3).

4.2 Effect of van Driest damping

The results show that the van Driest damping function is essential for accurate flow prediction in turbulent boundary layers, particularly near the wall. It helps capture the transition from viscous-dominated behaviour close to the wall, to fully turbulent behaviour farther away by reducing turbulence production in the Prandtl and $k - \epsilon$ models.

Comparing the cases with and without the van Driest damping functions, Figure 1 clearly shows how both damped models align better with the DNS velocity profile results across all values of y^+ and Re_τ . Furthermore, Figure 2 and Figure 3 show that this improvement is particularly effective in regions close to the wall, where accurate modelling is challenging due to sharp velocity gradients. Without the damping function, both the Prandtl and $k - \epsilon$ models overestimate the turbulent velocity fluctuations, leading to inaccurate velocity profiles and stress predictions. This is evident in the undamped profiles, which fail to capture the laminar-like behaviour of the flow within the viscous sublayer and overpredict turbulent Reynolds stresses near the wall.

4.3 Comparison of Models

The $k - \epsilon$ model and the Prandtl model differ in both implementation and results. The Prandtl model uses a mixing length hypothesis with an empirical damping function to capture the near-wall behaviour. The $k - \epsilon$ model, however, is a two-equation model that solves for the turbulent kinetic energy (k) and its dissipation rate (ϵ), allowing it to predict the effects of turbulence on the mean flow in a more detailed manner.

From the results in Figure 1, it can be observed that the $k - \epsilon$ model with damping provides a better match to DNS data than the Prandtl model for all Re_τ . The $k - \epsilon$ model's ability to solve for both k and ϵ makes it more versatile and capable of adapting to various flow conditions, whereas the Prandtl model's predictions are more limited by its reliance on a single mixing length. These differences become more evident in the buffer and logarithmic regions, where the Prandtl model begins to deviate more and more significantly from the DNS data as it cannot capture the turbulent behaviour as accurately.

5 Comparison with Laminar Flow

Throughout this investigation, it has been assumed that the flow being analysed is characterised by turbulent behaviour, which is why modelling is required to describe the flow. However, laminar channel flow has an analytical expression of streamwise flow velocity:

$$\bar{u}(y)_{laminar} = \frac{1}{2\mu} \cdot \frac{\partial \bar{P}}{\partial x} \cdot (y^2 - H^2) \quad (20)$$

Consider the graphs in Figure 4 showing the mean velocity profiles for the laminar and turbulent flows for different Reynolds numbers: $Re_\tau = 180$, $Re_\tau = 395$, and $Re_\tau = 590$. The profiles include the laminar analytical solution and turbulent predictions from the Prandtl

and $k - \epsilon$ models with van Driest damping. Note that $y = 0$ is taken to be at the channel's centerline, so the graphs show the velocity distribution for the top half of the channel.

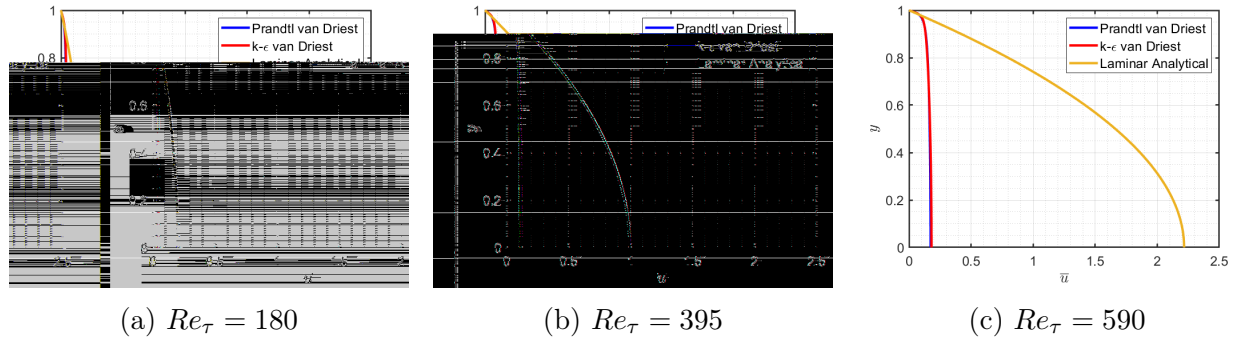


Figure 4: Mean velocity profile for laminar and turbulent flow

For the laminar flow, the velocity profile follows a parabolic shape, with the maximum velocity at the centre of the channel and zero velocity at the wall due to the no-slip condition. As Re_τ increases, the streamwise pressure gradient increases, and thus the maximum velocity increases significantly. In contrast, the turbulent profiles for both the Prandtl and $k - \epsilon$ models exhibit a more flattened shape compared to the laminar profile, with higher velocities near the wall. This characteristic is due to increased momentum transfer by turbulent eddies, which enhances mixing and reduces the velocity gradient in the core flow region.

As Re_τ increases, the friction velocity u_τ also increases, leading to a larger near-wall shear stress and more intense turbulence. This shift results in a greater deviation from the laminar profile, as the increase in velocity for the turbulent case is minimal in comparison.

6 Conclusion

The velocity profiles for $Re_\tau = 180$, $Re_\tau = 395$, and $Re_\tau = 590$ show that the Prandtl model is less accurate at low Re_τ , as seen in the logarithmic layer for $Re_\tau = 180$, but performs better at higher Re_τ . In contrast, the $k - \epsilon$ model fits well for $Re_\tau = 180$, but deviates more at $Re_\tau = 590$. This trend is also seen in the Reynolds stresses, which converge to similar $-uv_{RMS}$ values as y^+ increases at higher Re_τ . As Re_τ increases, steeper near-wall velocity gradients and stronger Reynolds stresses arise, requiring finer grid resolution near the wall. The need for grid refinement for these types of flows adds complexity to the numerical setup and increases computational cost, which could be further addressed by implementing non-uniform grid spacing to improve the accuracy of the models.

Furthermore, this investigation demonstrated that the van Driest damping function plays a critical role when regulating turbulence production in the Prandtl and $k - \epsilon$ models. This function helps prevent the overprediction of turbulent viscosity close to the wall, enabling a more accurate representation of the flow's laminar-like behaviour within the viscous sublayer. Thus, the van Driest damping function enhances model accuracy in predicting turbulent channel flow across varying Reynolds numbers by ensuring that the model better aligns with physical expectations in regions dominated by viscous forces.

List of Figures

1	Mean velocity profile: — Damped Prandtl/ $k-\epsilon$, — Undamped Prandtl/ $k-\epsilon$, — DNS; for $Re_\tau = 180$ (Row 1), $Re_\tau = 395$ (Row 2), $Re_\tau = 590$ (Row 3).	6
2	Reynolds shear stress $-\overline{u'v'}$ normalized by the wall shear velocity in wall coordinates — Damped Prandtl/ $k-\epsilon$, — Undamped Prandtl/ $k-\epsilon$, — DNS; for $Re_\tau = 180$ (Row 1), $Re_\tau = 395$ (Row 2), $Re_\tau = 590$ (Row 3).	7
3	Reynolds shear stress $-\overline{u'v'}$ normalized by the wall shear velocity in global coordinates — Damped Prandtl/ $k-\epsilon$, — Undamped Prandtl/ $k-\epsilon$, — DNS; for $Re_\tau = 180$ (Row 1), $Re_\tau = 395$ (Row 2), $Re_\tau = 590$ (Row 3).	8
4	Mean velocity profile for laminar and turbulent flow	10

List of Tables

1	Boundary conditions for k-epsilon model for the fully developed channel flow.	5
---	---	---

References

- [1] R. Moser, J. Kim, and P. Moin, “Direct numerical simulation of turbulent channel flow up to $re_\tau = 590$,” *Physics of Fluids*, 1999.
- [2] T. Kajishima and K. Taira, *Computational fluid mechanics: incompressible turbulent flows*. Place of publication not identified, 2018. OCLC: 1035427978.
- [3] H. K. Myong and N. Kasagi, “A new approach to the improvement of k-epsilon; turbulence model for wall-bounded shear flows,” *JSME international journal. Ser. 2, Fluids engineering, heat transfer, power, combustion, thermophysical properties*, vol. 33, no. 1, pp. 63–72, 1990.
- [4] B. M. Sumer and D. R. Fuhrman, *Turbulence in Coastal and Civil Engineering*, vol. 51 of *Advanced Series on Ocean Engineering*. WORLD SCIENTIFIC, Apr. 2020.
- [5] J. Kim, P. Moin, and R. Moser, “Turbulence statistics in fully developed channel flow at low Reynolds number,” *Journal of Fluid Mechanics*, vol. 177, pp. 133–166, Apr. 1987.

7 Appendix

7.1 Acknowledgments

In preparation for this report, AI/ChatGPT was used to plot the figures using a consistent format and to help with some of the formatting of the report.

7.2 Contributions

Marcos Garcia Bravo (s244059): Contribution to the derivation of equations and models used throughout the report, writing the introduction, governing equations, results and conclusion sections in the report.

Contribution: 33%

Jakub Kuleta (s244053): Coding of mesh for $k - \epsilon$, coding of $k - \epsilon$ model, contribution to the derivation of equations and models used throughout the report, writing the $k - \epsilon$ section in the report.

Contribution: 33%

Szczepan Zygmunt Letkiewicz (s244067): Coding Prandtl model, coding of $k - \epsilon$ model, writing the Prandtl section in the report.

Contribution: 33%

7.3 Code

```
1  clc;clear all;close all
2
3  %% Parameters
4  visc_nu = 1.56*10^(-5);
5  rho = 1.225;
6  visc_mu = visc_nu*rho;
7  H = 1;
8
9  %% DNS
10 Re_tau_values = [180 395 590];
11 U_tau = Re_tau_values.*visc_nu/H;
12
13 filenames = {'DNS_180.txt', 'DNS_395.txt', 'DNS_590.txt'};
14 DNSData = LoadDNSData(filenames);
15
16 %% Loop
17 for j = 1:length(Re_tau_values)
18
19     %% Prandalt
20     U_Prandalt = PrandaltSolver(Re_tau_values, visc_nu, H);
```



```

21
22     %% K-Epsilon
23     Re_tau = Re_tau_values(j);
24     [U_KEPSILON_damped(j), U_KEPSILON_undamped(j)] = KEPSILON(Re_tau,
25         visc_nu, rho, H);
26 end
27
28 %% Velocity plot
29 plotVelocityProfiles(Re_tau_values, DNSData, U_Prandalt, U_KEPSILON_damped,
30     U_KEPSILON_undamped, H, visc_nu);
31 %% Stress Plot
32 plotReynoldsStress(Re_tau_values, DNSData, visc_mu, U_Prandalt,
33     U_KEPSILON_damped, U_KEPSILON_undamped, H);
34 %% Model comp
35 plotVelocityComparison(Re_tau_values, DNSData, U_Prandalt, U_KEPSILON_damped
36     , H, visc_nu);
37 %% Turbulent vs Laminar
38 plotLaminarTrubulent(Re_tau_values, U_Prandalt, U_KEPSILON_damped, H,
39     visc_nu)

```

Listing 1: Main Script

```

1 function c = finiteDiffCoeff(x,k)
2
3     n = length(x);
4     A = ones(n,n);
5     xrow = x';
6
7     for i=2:n
8         A(i,:) = (xrow .^(i-1)) ./ factorial(i-1);
9     end
10
11     b = zeros(n,1);    % b is right hand side,
12     b(k+1) = 1;        % so k'th derivative term remains
13     c = A\b;           % solve system for coefficients
14
15     c = c';
16
17 end

```

Listing 2: Finite Difference Coefficients

```

1 function u_laminar = get_analytical_solution(y, DP_dx, visc_mu, H)
2     u_laminar = 1 / (2 * visc_mu) * DP_dx * (y.^2 - H^2);

```

3 `end`

Listing 3: Analytical Solution

```

1 function [U_kepsilon_damped, U_kepsilon_undamped] = kepsilon(Re_tau, visc_nu
  , rho, H)
2     %% Mesh
3     N = 1000;
4     dy = H / (N - 1);
5     MESH = meshGenerator(H, N);
6     U_tau = Re_tau * visc_nu / H;
7     y_Plus = MESH.y * U_tau / visc_nu;
8
9     %% Damping function
10    f_u_func_damped = @(y_Plus, A_Plus) (1 - exp(-y_Plus / A_Plus)).^2;
11    f2_func_damped = @(Re_tau, y_Plus) (1 - 2/9 * exp(-(Re_tau / 6).^2)) .*
        (1 - exp(-y_Plus / 5)).^2;
12
13    f_u_func_undamped = @(y_Plus, A_Plus) (y_Plus * A_Plus * 0 + 1);
14    f2_func_undamped = @(Re_tau, y_Plus) (y_Plus * Re_tau * 0 + 1);
15
16    %% Solver
17    % Explicit method — is unstable needs to be changed
18    % initial guess damped
19    visc_nu_eddy = zeros(N, 1);
20    U = 180*ones(N, 1);
21    k = 180*ones(N, 1);
22    epsilon = 180*ones(N, 1);
23    alpha_U = 1;
24    alpha_k = 1;
25    alpha_epsilon = 1;
26
27    %solve
28    U_damped_struct = KEpsilonSolver(Re_tau, visc_nu, rho, H,
        f_u_func_damped, f2_func_damped, visc_nu_eddy, U, k, epsilon, alpha_U,
        alpha_k, alpha_epsilon, MESH, dy, N);
29
30    %% Struct damped
31    U_kepsilon_damped.Re_tau = Re_tau;
32    U_kepsilon_damped.y_Plus = y_Plus;
33    U_kepsilon_damped.U = U_damped_struct.U;
34    U_kepsilon_damped.k = U_damped_struct.k;
35    U_kepsilon_damped.epsilon = U_damped_struct.epsilon;
36    U_kepsilon_damped.visc_nu_eddy = U_damped_struct.visc_nu_eddy;

```



```

37
38 % initial guess undamped
39 N = 2000;
40 dy = H / (N - 1);
41 MESH = meshGenerator(H, N);
42 y_Plus = MESH.y * U_tau / visc_nu;
43 visc_nu_eddy = zeros(N, 1);
44 U = 200*ones(N, 1);
45 k = 200*ones(N, 1);
46 epsilon = 200*ones(N, 1);
47 alpha_U = 1;
48 alpha_k = 1;
49 alpha_epsilon = 1;
50 % solve
51 U_undamped_struct = KEpsilonSolver(Re_tau, visc_nu, rho, H,
    f_u_func_undamped, f2_func_undamped, visc_nu_eddy, U, k, epsilon, alpha_U,
    alpha_k, alpha_epsilon, MESH, dy, N);
52
53 % Re_tau 180 Converges for Setting: N = 1500; initial values 200;
    Relaxation factor 1
54 % Re_tau 395 Sor of converges for: N = 1700; Initial values 300:
    Relaxation factor 1
55 % Re_tau 590 Converges for Setting: N = 1000; initial values 200;
    Relaxation factor 1
56
57 %% Struct undamped
58 U_kepsilon_undamped.Re_tau = Re_tau;
59 U_kepsilon_undamped.y_Plus = y_Plus;
60 U_kepsilon_undamped.U = U_undamped_struct.U;
61 U_kepsilon_undamped.k = U_undamped_struct.k;
62 U_kepsilon_undamped.epsilon = U_undamped_struct.epsilon;
63 U_kepsilon_undamped.visc_nu_eddy = U_undamped_struct.visc_nu_eddy;
64 end

```

Listing 4: K-Epsilon Model

```

1 function U_output = KEpsilonSolver(Re_tau, visc_nu, rho, H, f_u_func,
    f2_func, visc_nu_eddy, U, k, epsilon, alpha_U, alpha_k, alpha_epsilon, MESH, dy, N)
2
3 %% Mesh
4 y = MESH.y;
5 ddy = MESH.ddy;
6 d2dy2 = MESH.d2dy2;
7

```

```

8      %% Constants
9      A_Plus = 26;
10     kappa = 0.41;
11
12     %%k-epslion
13     C_mu = 0.09;
14     C_1 = 1.44;
15     C_2 = 1.92;
16     sigma_k = 1;
17     sigma_epsilon = 1.3;
18
19     U_tau = Re_tau .* visc_nu / H;
20     DP_dx = -rho * Re_tau^2 * visc_nu^2 / H^3;
21
22     %% Boundary conditions
23     % refernce
24     visc_nu_eddy(1) = 0;
25     visc_nu_eddy(end) = visc_nu_eddy(end - 1);
26     k(1) = 0;
27     k(end) = k(end - 1);
28     epsilon(1) = 0;
29     epsilon(end) = epsilon(end - 1);
30     U(1) = 0;
31     U(end) = U(end - 1);
32
33     %% Solver
34     maxiter = 500;
35     tolerance = 1e-6;
36     converged = false;
37
38     U_kepsilon = struct();
39     for i = 1:maxiter
40         U_old = U;
41         k_old = k;
42         epsilon_old = epsilon;
43
44         dUdy = ddy * U;
45         y_Plus = y*U_tau/visc_nu;
46
47         f_u = f_u_func(y_Plus, A_Plus);
48         f2 = f2_func(Re_tau, y_Plus);
49
50         %% Epsilon loop
51         % initialize

```

```

52     visc_nu_eddy = f_u.* C_mu .* (k.^2) ./ epsilon;
53
54     % Pk
55     Pk = visc_nu_eddy .* dUdy.^2;
56     visc_term = visc_nu + visc_nu_eddy / sigma_epsilon;
57
58     % A epsilon
59     A_epsilon = bsxfun(@times, visc_term, d2dy2) + bsxfun(@times, ddy *
        visc_term, ddy);
60     A_epsilon(1, :) = 0;
61     A_epsilon(1, 1) = 1;
62     A_epsilon(end, :) = 0;
63     A_epsilon(end, end - 1) = -1;
64     A_epsilon(end, end) = 1;
65
66     for index = 2:N-1
67         A_epsilon(index, index) = A_epsilon(index, index) - C_2* f2(
            index) * epsilon(index) / max(k(index), 1e-10);
68     end
69
70     % B epsilon
71     B_epsilon = -C_1 * epsilon ./ max(k, 1e-10) .* Pk;
72     B_epsilon(1) = visc_nu * (k(1) - 2 * k(2) + k(3)) / dy^2;
73     B_epsilon(end) = 0;
74
75     % Solve epsilon
76     epsilon_new = linsolve(A_epsilon, B_epsilon);
77     epsilon_new = max(epsilon_new, 1e-8);
78     epsilon = alpha_epsilon * epsilon_new + (1 - alpha_epsilon) *
        epsilon_old;
79
80     %% k loop
81     visc_nu_eddy = f_u .* C_mu .* k.^2 ./ epsilon;
82     visc_term = visc_nu + visc_nu_eddy / sigma_k;
83
84     % Matrix k
85     A_k = bsxfun(@times, visc_term, d2dy2) + bsxfun(@times, ddy *
        visc_term, ddy);
86     A_k(1, :) = 0;
87     A_k(1, 1) = 1;
88     A_k(end, :) = 0;
89     A_k(end, end - 1) = -1;
90     A_k(end, end) = 1;
91

```

```

92     for j = 2:N-1
93         A_k(j, j) = A_k(j, j) - epsilon(j) / k(j);
94     end
95
96     % B matrix
97     B_k = -Pk;
98     B_k(1) = 0;
99     B_k(end) = 0;
100
101     % Solve k
102     k_new = linsolve(A_k, B_k);
103     k_new = max(k_new, 1e-8);
104     k = alpha_k * k_new + (1 - alpha_k) * k_old;
105
106     %% U loop
107     visc_nu_eddy = f_u.* C_mu .* k.^2 ./ epsilon;
108     visc_term = visc_nu + visc_nu_eddy;
109
110     % A U
111     A_U = bsxfun(@times, visc_term, d2dy2) + bsxfun(@times, ddy *
        visc_term, ddy);
112     A_U(1, :) = 0;
113     A_U(1, 1) = 1;
114     A_U(end, :) = 0;
115     A_U(end, end - 1) = -1;
116     A_U(end, end) = 1;
117
118     % B U
119     B_U = ones(N, 1) * 1 / rho * DP_dx;
120     B_U(1) = 0;
121     B_U(end) = 0;
122
123     % Solve U
124     U_new = linsolve(A_U, B_U);
125     U = alpha_U * U_new + (1 - alpha_U) * U_old;
126
127     %% Convergence
128     residual = norm(U - U_old);
129     residual_history(mod(i-1, 10) + 1) = residual; % Store the latest
        residual
130     disp(['Iteration: ', num2str(i), ' Residual: ', num2str(residual)]);
131
132     if residual < tolerance
133         converged = true;

```

```

134         disp('Converged');
135         break;
136     end
137
138     if i > 10 && max(residual_history) - min(residual_history) < 1e-8
139         converged = true;
140         disp('Converged');
141         break;
142     end
143 end
144 if ~converged
145     warning('Solution did not converge');
146 end
147 U_output = struct();
148 U_output.U = U;
149 U_output.k = k;
150 U_output.epsilon = epsilon;
151 U_output.visc_nu_eddy = visc_nu_eddy;
152 end

```

Listing 5: K-Epsilon Solver

```

1 function DNSData = LoadDNSData(filenamees)
2     DNSData = struct();
3     Re_tau_values = [180, 395, 590];
4
5     for i = 1:length(filenamees)
6         data = readmatrix(filenamees{i});
7         DNSData.(['y_', num2str(Re_tau_values(i))]) = data(:, 1);
8         DNSData.(['y_plus_', num2str(Re_tau_values(i))]) = data(:, 2); %
9             Column y
10        DNSData.(['U_mean_', num2str(Re_tau_values(i))]) = data(:, 3); %
11            column y^+
12        DNSData.(['DU_dy_', num2str(Re_tau_values(i))]) = data(:, 4); %
13            column du/dy
14        DNSData.(['R_uu_', num2str(Re_tau_values(i))]) = data(:, 5); %
15            Column R_uu
16        DNSData.(['R_vv_', num2str(Re_tau_values(i))]) = data(:, 6); %
17            Column R_vv
18        DNSData.(['R_wv_', num2str(Re_tau_values(i))]) = data(:, 7); %
19            Column R_wv
20        DNSData.(['R_uv_', num2str(Re_tau_values(i))]) = data(:, 8); %
21            Column R_uv
22        DNSData.(['R_uw_', num2str(Re_tau_values(i))]) = data(:, 9); %

```

```

16         Column = R_uw
           DNSData.(['R_vw_', num2str(Re_tau_values(i))]) = data(:, 10); %
           Column R_vw
17     end
18 end

```

Listing 6: DNS Data Loader

```

1 function [MESH] = meshGenerator(H, n)
2     %% length
3     ns = 3;
4     y = linspace(0, H, n)';
5
6     %% derivative
7     dy = y(2) - y(1);
8     ddy = zeros(n, n);
9
10    ddy(1,1:7) = finiteDiffCoeff(0:6, 1);
11    ddy(2,1:7) = finiteDiffCoeff(-1:5, 1);
12    ddy(3,1:7) = finiteDiffCoeff(-2:4, 1);
13    ddy(n-2,n-6:n) = finiteDiffCoeff(-4:2, 1);
14    ddy(n-1,n-6:n) = finiteDiffCoeff(-5:1, 1);
15    ddy(n, n-6:n) = finiteDiffCoeff(-6:0, 1);
16
17    for i=1+ns:n-ns
18        ddy(i,:) = 0.0;
19        ddy(i,i-ns:i+ns) = finiteDiffCoeff(-ns:ns, 1);
20    end
21
22    ddy = ddy / dy;
23
24    %% 2nd derivative
25    d2dy2 = zeros(n, n);
26
27    d2dy2(1,1:7) = finiteDiffCoeff(0:6, 2);
28    d2dy2(2,1:7) = finiteDiffCoeff(-1:5, 2);
29    d2dy2(3,1:7) = finiteDiffCoeff(-2:4, 2);
30    d2dy2(n-2,n-6:n) = finiteDiffCoeff(-4:2, 2);
31    d2dy2(n-1,n-6:n) = finiteDiffCoeff(-5:1, 2);
32    d2dy2(n, n-6:n) = finiteDiffCoeff(-6:0, 2);
33
34    for i=1+ns:n-ns
35        d2dy2(i,:) = 0.0;
36        d2dy2(i,i-ns:i+ns) = finiteDiffCoeff(-ns:ns, 2);

```

```
37     end
38
39     d2dy2 = d2dy2 / dy^2;
40
41     %% output
42     MESH = struct('y', y, 'ddy', ddy, 'd2dy2', d2dy2);
```

```

33     plot(u_laminar,y,'DisplayName', 'Laminar Analytical', 'LineWidth',
        lineWidth);
34
35     %% appearance — Chatgpt
36     ax = gca; % Get current axes
37     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
38     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
39     ax.Box = 'on'; % Add a box around the plot
40
41     grid on;
42     grid minor; % Enable minor grid lines for better visibility
43     ax.GridLineStyle = '-'; % Solid grid lines
44     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
45
46     % Set font size for better visibility
47     ax.FontSize = 14; % Increase font size for axes
48     xlabel('$\overline{u}$', 'Interpreter', 'latex', 'FontSize', 16); %
        Increase label font size
49     ylabel('$y$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
        label font size
50
51     % Display legend without vertical lines
52     legend('show', 'Location', 'northeast', 'FontSize', 14); % Increase
        legend font size
53     hold off;
54
55     % Set x-axis limits
56     xlim([0 2.5]);
57     hold off;
58     end
59 end

```

Listing 8: Laminar and Turbulent Plotter

```

1 function plotReynoldsStress(Re_tau_values,DNSData, visc_nu, U_Prandalt,
    U_kepsilon_damped, U_kepsilon_undamped,H);
2     %% Constants
3     H = 1;
4     A_Plus = 26;
5     kappa = 0.41;
6     lineWidth = 2.5;
7     numReTau = length(Re_tau_values);
8
9     %% Reynolds Stress calc

```



```

10     for j = 1:numReTau
11         U_tau = Re_tau_values(j) * visc_nu / H;
12         height_max = max(DNSData.(['y_', num2str(Re_tau_values(j))]]));
13
14         %% Prandalt
15         U_Prandalt_damped = U_Prandalt(j).U_damped;
16         U_Prandalt_undamped = U_Prandalt(j).U_undamped;
17
18         %% Mesh
19         y = linspace(0, H, 1000)';
20         y_Prandalt_Plus = y * U_tau / visc_nu;
21
22         % Normalize velocities
23         U_Prandalt_Plus_damped = U_Prandalt_damped / U_tau;
24         U_Prandalt_Plus_undamped = U_Prandalt_undamped / U_tau;
25
26         %% Prandtl Stress Calculation
27         % For damped model
28         du_dy_exp_damped = gradient(U_Prandalt_damped, y);
29         l_t_damped = -(kappa * y .* (1 - exp(-y_Prandalt_Plus / A_Plus)));
30         Prandalt_Reynolds_stress_damped = -l_t_damped.^2 .* abs(
31             du_dy_exp_damped).^2;
32         Prandalt_uv_rms_damped = sqrt(-Prandalt_Reynolds_stress_damped) /
33             U_tau;
34
35         % For undamped model
36         du_dy_exp_undamped = gradient(U_Prandalt_undamped, y);
37         l_t_undamped = kappa * y;
38         Prandalt_Reynolds_stress_undamped = -l_t_undamped.^2 .* abs(
39             du_dy_exp_undamped).^2;
40         Prandalt_uv_rms_undamped = sqrt(-Prandalt_Reynolds_stress_undamped)
41             / U_tau;
42
43         %% full channel depth y/2H
44         figure;
45         plot(DNSData.(['y_', num2str(Re_tau_values(j))])/height_max-1, -
46             DNSData.(['R_uv_', num2str(Re_tau_values(j))]]), 'k—', '
47             DisplayName', 'DNS', 'LineWidth', lineWidth);
48         hold on;
49         h1 = plot(y/H-1, Prandalt_uv_rms_undamped, 'DisplayName', 'Prandtl
50             model', 'LineWidth', lineWidth);
51         color_undamped = get(h1, 'Color');
52         h2 = plot(y/H-1, Prandalt_uv_rms_damped, 'DisplayName', 'van Driest
53             damping', 'LineWidth', lineWidth);

```

```

46     color_damped = get(h2, 'Color');
47     % flip side mirror
48     plot(-flipud(fliplr(DNSData.(['y_', num2str(Re_tau_values(j))]/
        height_max))+1, -flipud(fliplr(-DNSData.(['R_uv_', num2str(
        Re_tau_values(j))])))), 'k—', 'LineWidth', lineWidth, '
        HandleVisibility', 'off');
49     plot(y/H, -flipud(fliplr(Prandalt_uv_rms_undamped)), 'LineWidth',
        lineWidth, 'HandleVisibility', 'off', 'Color', color_undamped);
50     plot(y/H, -flipud(fliplr(Prandalt_uv_rms_damped)), 'LineWidth',
        lineWidth, 'HandleVisibility', 'off', 'Color', color_damped);
51     plot([-1 1], [1 -1], 'b', 'DisplayName', '$\frac{\tau}{\tau_w}$', '
        LineWidth', lineWidth*0.75);
52     yline(0, 'k—', 'LineWidth', 1, 'HandleVisibility', 'off');
53     grid on;
54     ylabel('-uv_{rms}');
55     xlabel('y/H');
56     legend('show', 'Interpreter', 'latex');
57     ylim([-1 1]);
58     hold off;
59
60     %% appearance —Chatgpt
61     ax = gca; % Get current axes
62     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
63     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
64     ax.Box = 'on'; % Add a box around the plot
65
66     grid on;
67     grid minor; % Enable minor grid lines for better visibility
68     ax.GridLineStyle = '-'; % Solid grid lines
69     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
70
71     % Set font size for better visibility
72     ax.FontSize = 14; % Increase font size for axes
73     xlabel('y/H', 'FontSize', 16); % Increase label font size
74     ylabel('-uv_{rms}', 'FontSize', 16); % Increase label font size
75
76     % Display legend with LaTeX interpretation
77     legend('show', 'Location', 'northwest', 'FontSize', 14, 'Interpreter
        ', 'latex'); % Increase legend font size
78     hold off;
79
80     % Set figure size
81     fig = gcf; % Get current figure
82     fig.Position(3) = 1200; % Set width of the figure (in pixels)

```

```
83 |         fig.Position(4) = 600; % Set height of the figure (in
```

```

122     fig.Position(4) = 600; % Set height of the figure (in pixels)
123
124     %% Kepsilon
125     U_kepsilon_damped_current = U_kepsilon_damped(j).U;
126     U_kepsilon_undamped_current = U_kepsilon_undamped(j).U;
127     k_kepsilon_damped_current = U_kepsilon_damped(j).k;
128     k_kepsilon_undamped_current = U_kepsilon_undamped(j).k;
129     epsilon_kepsilon_damped_current = U_kepsilon_damped(j).epsilon;
130     epsilon_kepsilon_undamped_current = U_kepsilon_undamped(j).epsilon;
131     C_mu = 0.09;
132
133     %% Mesh
134     y_kepsilon_Plus_undamped = U_kepsilon_undamped(j).y_Plus;
135     y_kepsilon_Plus_damped = U_kepsilon_damped(j).y_Plus;
136     y_kepsilon_undamped = y_kepsilon_Plus_undamped * visc_nu / U_tau;
137     y_kepsilon_damped = y_kepsilon_Plus_damped * visc_nu / U_tau;
138
139     %% Prandtl Stress Calculation
140     % For damped model
141     du_dy_exp_damped = gradient(U_kepsilon_damped_current,
142                                y_kepsilon_damped);
143     f_u_damped = (1 - exp(-y_kepsilon_Plus_damped / A_Plus)).^2;
144     kepsilon_Reynolds_stress_damped = -(f_u_damped.*C_mu.*
145                                         k_kepsilon_damped_current.^2./epsilon_kepsilon_damped_current) .*
146                                         abs(du_dy_exp_damped);
147     kepsilon_uv_rms_damped = sqrt(-kepsilon_Reynolds_stress_damped) /
148                                U_tau;
149
150     % For undamped model
151     du_dy_exp_undamped = gradient(U_kepsilon_undamped_current,
152                                  y_kepsilon_undamped);
153     f_u_undamped = (1 - y_kepsilon_Plus_undamped * 0);
154     kepsilon_Reynolds_stress_undamped = -(f_u_undamped.*C_mu.*
155                                         k_kepsilon_undamped_current.^2./epsilon_kepsilon_undamped_current
156                                         ) .* abs(du_dy_exp_undamped);
157     kepsilon_uv_rms_undamped = sqrt(-kepsilon_Reynolds_stress_undamped)
158                                / U_tau;
159
160     %% full channel depth y/2H
161     figure;
162     plot(DNSData(['y_', num2str(Re_tau_values(j))])/height_max-1, —
163          DNSData(['R_uv_', num2str(Re_tau_values(j))]), 'k—', '
164              DisplayName', 'DNS', 'LineWidth', lineWidth);
165     hold on;

```

```

156     h1 = plot(y_kepsilon_undamped/H-1, kepsilon_uv_rms_undamped, '
        DisplayName', '$k-\epsilon$', 'LineWidth', lineWidth);
157     color_undamped = get(h1, 'Color');
158     h2 = plot(y_kepsilon_damped/H-1, kepsilon_uv_rms_damped, '
        DisplayName', '$k-\epsilon$ van Driest', 'LineWidth', lineWidth);
159     color_damped = get(h2, 'Color');
160     % flip side mirror
161     plot(-flipud(fliplr(DNSData.(['y_', num2str(Re_tau_values(j))])/
        height_max))+1, -flipud(fliplr(-DNSData.(['R_uv_', num2str(
        Re_tau_values(j))])), 'k—', 'LineWidth', lineWidth, '
        HandleVisibility', 'off');
162     plot(y_kepsilon_undamped/H, -flipud(fliplr(kepsilon_uv_rms_undamped)
        ), 'LineWidth', lineWidth, 'HandleVisibility', 'off', 'Color',
        color_undamped);
163     plot(y_kepsilon_damped/H, -flipud(fliplr(kepsilon_uv_rms_damped)), '
        LineWidth', lineWidth, 'HandleVisibility', 'off', 'Color',
        color_damped);
164     plot([-1 1], [1 -1], 'b', 'DisplayName', '$\frac{\tau}{\tau_w}$', '
        LineWidth', lineWidth*0.75);
165     yline(0, 'k—', 'LineWidth', 1, 'HandleVisibility', 'off');
166     grid on;
167     ylabel('-uv_{rms}');
168     xlabel('y/H');
169     legend('show', 'Interpreter', 'latex');
170     ylim([-1 1]);
171     hold off;
172
173     %% Enhance grid and axes appearance
174     ax = gca; % Get current axes
175     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
176     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
177     ax.Box = 'on'; % Add a box around the plot
178
179     grid on;
180     grid minor; % Enable minor grid lines for better visibility
181     ax.GridLineStyle = '-'; % Solid grid lines
182     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
183
184     % Set font size for better visibility
185     ax.FontSize = 14; % Increase font size for axes
186     xlabel('y/H', 'FontSize', 16); % Increase label font size
187     ylabel('-uv_{rms}', 'FontSize', 16); % Increase label font size
188
189     % Display legend with LaTeX interpretation

```

```

190     legend('show', 'Location', 'northwest', 'FontSize', 14, 'Interpreter', 'latex'); % Increase legend font size
191     hold off;
192
193     % Set figure size
194     fig = gcf; % Get current figure
195     fig.Position(3) = 1200; % Set width of the figure (in pixels)
196     fig.Position(4) = 600; % Set height of the figure (in pixels)
197
198     %% Figure near wall region
199     figure;
200     plot(DNSData(['y_plus_', num2str(Re_tau_values(j))])/height_max, -
          DNSData(['R_uv_', num2str(Re_tau_values(j))]), 'k—', '
          DisplayName', 'DNS', 'LineWidth', lineWidth);
201     hold on;
202     plot(y_kepsilon_Plus_undamped, kepsilon_uv_rms_undamped, '
          DisplayName', 'k-\epsilon', 'LineWidth', lineWidth);
203     plot(y_kepsilon_Plus_damped, kepsilon_uv_rms_damped, 'DisplayName',
          'k-\epsilon van Driest', 'LineWidth', lineWidth);
204     grid on;
205     ylabel('-uv_{rms}');
206     xlabel('y^+');
207     xlim([0 100]);
208     ylim([0 1]);
209     legend('show', 'Location', 'southeast');
210     hold off;
211
212     %% Enhance grid and axes appearance
213     ax = gca; % Get current axes
214     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
215     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
216     ax.Box = 'on'; % Add a box around the plot
217
218     grid on;
219     grid minor; % Enable minor grid lines for better visibility
220     ax.GridLineStyle = '-'; % Solid grid lines
221     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
222
223     % Set font size for better visibility
224     ax.FontSize = 14; % Increase font size for axes
225     xlabel('y^+', 'FontSize', 16); % Increase label font size
226     ylabel('-uv_{rms}', 'FontSize', 16); % Increase label font size
227
228     % Display legend with LaTeX interpretation

```

```
229     legend('show', 'Location', 'southeast', 'FontSize', 14, 'Interpreter', 'latex'); % Increase legend font size
230     hold off;
231
232     % Set figure size
233     fig = gcf; % Get current figure
234     fig.Position(3) = 1200; % Set width of the figure (in pixels)
235     fig.Position(4) = 600; % Set height of the figure (in pixels)
236 end
237 end
```

Listing 9: Reynolds Stress Plotter

```
1 function plotVelocityComparison(Re_tau_values, DNSData, U
```

```

31     %% apperance — Chatgpt
32     y_viscous = 5;
33     y_buffer = 30;
34     y_log = 0.3 * H * U_tau / visc_nu; % Adjust log layer according to
        y/H
35
36     % Plot vertical lines (without adding to legend)
37     xline(y_viscous, 'k-', 'y^+ = 5', 'LabelOrientation', 'horizontal',
        ...
38         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
        HandleVisibility', 'off', 'FontSize', 16);
39     xline(y_buffer, 'k-', 'y^+ = 30', 'LabelOrientation', 'horizontal',
        ...
40         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
        HandleVisibility', 'off', 'FontSize', 16);
41     xline(y_log, 'k-', 'y^+ = 0.3y/H', 'LabelOrientation', 'horizontal',
        ...
42         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
        HandleVisibility', 'off', 'FontSize', 16);
43
44     %% Enhance grid and axes appearance
45     ax = gca; % Get current axes
46     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
47     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
48     ax.Box = 'on'; % Add a box around the plot
49
50     grid on;
51     grid minor; % Enable minor grid lines for better visibility
52     ax.GridLineStyle = '-'; % Solid grid lines
53     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
54
55     % Set font size for better visibility
56     ax.FontSize = 14; % Increase font size for axes
57     xlabel('$y^+$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
        label font size
58     ylabel('$U^+$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
        label font size
59
60     % Display legend without vertical lines
61     legend('show', 'Location', 'northwest', 'FontSize', 14); % Increase
        legend font size
62     hold off;
63
64     % Set x-axis limits

```



```

65     xlim([10−1 103]);
66
67     % Set semilog scale on the x-axis
68     set(gca, 'XScale', 'log');
69
70     fig = gcf; % Get current figure
71     fig.Position(3) = 1200; % Set width of the figure (in pixels)
72     fig.Position(4) = 600; % Set height of the figure (in pixels)
73     hold off;
74 end
75 end

```

Listing 10: Velocity Comparison Plotter

```

1 function plotVelocityProfiles(Re_tau_values, DNSData, U_Prandalt,
   U_kepsilon_damped, U_kepsilon_undamped, H, visc_nu)
2     lineWidth = 2.5;
3     numReTau = length(Re_tau_values);
4     for j = 1:numReTau
5         U_tau = Re_tau_values(j) * visc_nu / H;
6         %% Load Data
7         % Mesh
8         dy = 0.001;
9         y = linspace(0, H, 1000)';
10        N = length(y);
11        U_Prandalt_damped = U_Prandalt(j).U_damped;
12        U_Prandalt_undamped = U_Prandalt(j).U_undamped;
13
14        %% Normalize Prandalt
15        y_Prandalt_Plus(:, j) = y * U_tau / visc_nu;
16        U_Prandalt_Plus_undamped(:, j) = U_Prandalt_undamped / U_tau;
17        U_Prandalt_Plus_damped(:, j) = U_Prandalt_damped / U_tau;
18
19        %% Plot prandalt
20        figure;
21        hold on;
22        semilogx(y_Prandalt_Plus(:, j), U_Prandalt_Plus_undamped(:, j), ...
23                'DisplayName', 'Prandtl', ...
24                'LineWidth', lineWidth);
25        semilogx(y_Prandalt_Plus(:, j), U_Prandalt_Plus_damped(:, j), ...
26                'DisplayName', 'Prandalt van Driest', ...
27                'LineWidth', lineWidth);
28        semilogx(DNSData.(['y_plus_', num2str(Re_tau_values(j))]), ...
29                DNSData.(['U_mean_', num2str(Re_tau_values(j))]), ...

```

```

30         'k—', 'DisplayName', 'DNS', ...
31         'LineWidth', lineWidth);
32
33     %% Figure — Format from ChatGPT
34     % Add vertical lines for layer boundaries without adding to the
      legend
35     y_viscous = 5;
36     y_buffer = 30;
37     y_log = 0.3 * H * U_tau / visc_nu; % Adjust log layer according to
      y/H
38
39     % Plot vertical lines (without adding to legend)
40     xline(y_viscous, 'k—', 'y^+ = 5', 'LabelOrientation', 'horizontal',
      ...
41         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
      HandleVisibility', 'off', 'FontSize', 16);
42     xline(y_buffer, 'k—', 'y^+ = 30', 'LabelOrientation', 'horizontal',
      ...
43         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
      HandleVisibility', 'off', 'FontSize', 16);
44     xline(y_log, 'k—', 'y^+ = 0.3y/H', 'LabelOrientation', 'horizontal',
      ...
45         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
      HandleVisibility', 'off', 'FontSize', 16);
46
47     % Enhance grid and axes appearance
48     ax = gca; % Get current axes
49     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
50     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
51     ax.Box = 'on'; % Add a box around the plot
52
53     grid on;
54     grid minor; % Enable minor grid lines for better visibility
55     ax.GridLineStyle = '-'; % Solid grid lines
56     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
57
58     % Set font size for better visibility
59     ax.FontSize = 14; % Increase font size for axes
60     xlabel('$y^+$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
      label font size
61     ylabel('$U^+$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
      label font size
62
63     % Display legend without vertical lines

```

```

64     legend('show', 'Location', 'northwest', 'FontSize', 14); % Increase
        legend font size
65     hold off;
66
67     % Set x-axis limits
68     xlim([10−1 103]);
69
70     % Set semilog scale on the x-axis
71     set(gca, 'XScale', 'log');
72
73     fig = gcf; % Get current figure
74     fig.Position(3) = 1200; % Set width of the figure (in pixels)
75     fig.Position(4) = 600; % Set height of the figure (in pixels)
76     hold off;
77
78     %% Load Data
79     % Mesh
80     U_kepsilon_damped_current = U_kepsilon_damped(j).U;
81     U_kepsilon_undamped_current = U_kepsilon_undamped(j).U;
82
83     %% Normalize Kepsilon
84     y_kepsilon_Plus_undamped(:, j) = U_kepsilon_undamped(j).y_Plus;
85     y_kepsilon_Plus_damped(:, j) = U_kepsilon_damped(j).y_Plus;
86     U_kepsilon_Plus_undamped(:, j) = U_kepsilon_undamped_current / U_tau
        ;
87     U_kepsilon_Plus_damped(:, j) = U_kepsilon_damped_current / U_tau;
88
89     %% Plot prandalt
90     figure;
91     hold on;
92     semilogx(y_kepsilon_Plus_undamped(:, j), U_kepsilon_Plus_undamped(:,
        j), ...
93         'DisplayName', 'k −\epsilon', ...
94         'LineWidth', lineWidth);
95     semilogx(y_kepsilon_Plus_damped(:, j), U_kepsilon_Plus_damped(:, j),
        ...
96         'DisplayName', 'k −\epsilon van Driest', ...
97         'LineWidth', lineWidth);
98     semilogx(DNSData(['y_plus_', num2str(Re_tau_values(j))]), ...
99         DNSData(['U_mean_', num2str(Re_tau_values(j))]), ...
100         'k—', 'DisplayName', 'DNS', ...
101         'LineWidth', lineWidth);
102
103     %% Figure — Format from ChatGPT

```

```

104     % Add vertical lines for layer boundaries without adding to the
        legend
105     y_viscous = 5;
106     y_buffer = 30;
107     y_log = 0.3 * H * U_tau / visc_nu; % Adjust log layer according to
        y/H
108
109     % Plot vertical lines (without adding to legend)
110     xline(y_viscous, 'k-', 'y^+ = 5', 'LabelOrientation', 'horizontal',
        ...
111         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
        HandleVisibility', 'off', 'FontSize', 16);
112     xline(y_buffer, 'k-', 'y^+ = 30', 'LabelOrientation', 'horizontal',
        ...
113         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
        HandleVisibility', 'off', 'FontSize', 16);
114     xline(y_log, 'k-', 'y^+ = 0.3y/H', 'LabelOrientation', 'horizontal',
        ...
115         'LabelVerticalAlignment', 'bottom', 'LineWidth', 1.5, '
        HandleVisibility', 'off', 'FontSize', 16);
116
117     % Enhance grid and axes appearance
118     ax = gca; % Get current axes
119     ax.XAxis.LineWidth = 1.5; % Thicker x-axis
120     ax.YAxis.LineWidth = 1.5; % Thicker y-axis
121     ax.Box = 'on'; % Add a box around the plot
122
123     grid on;
124     grid minor; % Enable minor grid lines for better visibility
125     ax.GridLineStyle = '-'; % Solid grid lines
126     ax.MinorGridLineStyle = ':'; % Dotted minor grid lines
127
128     % Set font size for better visibility
129     ax.FontSize = 14; % Increase font size for axes
130     xlabel('$y^+$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
        label font size
131     ylabel('$U^+$', 'Interpreter', 'latex', 'FontSize', 16); % Increase
        label font size
132
133     % Display legend without vertical lines
134     legend('show', 'Location', 'northwest', 'FontSize', 14); % Increase
        legend font size
135     hold off;
136

```

```

137     % Set x-axis limits
138     xlim([10−1 103]);
139
140     % Set semilog scale on the x-axis
141     set(gca, 'XScale', 'log');
142
143     fig = gcf; % Get current figure
144     fig.Position(3) = 1200; % Set width of the figure (in pixels)
145     fig.Position(4) = 600; % Set height of the figure (in pixels)
146     hold off;
147 end
148 end

```

Listing 11: Velocity Profiles Plotter

```

1 function U_Prandalt = PrandaltSolver(Re_tau_values, visc_nu, H)
2
3 %% Constants
4 A_Plus = 26;
5 kappa = 0.41;
6
7 U_Prandalt = struct();
8
9 %% Mesh
10 dy = 0.001;
11 y = linspace(0, H, 1000)';
12 N = length(y);
13
14 for j = 1:length(Re_tau_values)
15
16     %% Re_tau, U_tau parameters
17     Re_tau = Re_tau_values(j);
18     U_tau = Re_tau * visc_nu / H;
19     DP_dx = −Re_tau2 * visc_nu2 / H3;
20
21     % Y plus
22     y_Plus = y * U_tau / visc_nu;
23
24     %% damping loop
25     model_types = {'damped', 'undamped'};
26     U_Output = zeros(N, 2);
27
28     for model_index = 1:length(model_types)
29         model_type = model_types{model_index};

```

```

30
31     %% Solver
32     A = zeros(N, N);
33
34     % type
35     if strcmp(model_type, 'damped')
36         l_t = kappa * y .* (1 - exp(-y_Plus / A_Plus));
37     else
38         l_t = kappa * y;
39     end
40
41     %% Find Matrix B
42     inside_sqrt = visc_nu^2 - 4 * DP_dx * (H - y) .* (l_t).^2;
43     B = (-visc_nu + sqrt(inside_sqrt)) ./ (2 * (l_t).^2) .* dy;
44
45     %% Forward Scheme
46     for i = 2:N-1
47         A(i, i-1) = -1;    % coeff U(y-dy)
48         A(i, i) = 1;      % coeff U(y)
49         A(i, i+1) = 0;    % coeff U(y+dy)
50     end
51
52     %% BC
53     % B - Matrix
54     B(1) = 0;
55     B(end) = 0;
56
57     % A - Matrix
58     A(1, 1) = 1;
59     A(1, 2) = 0;
60     A(end, end-1) = -1;
61     A(end, end) = 1;
62
63     %% Solve
64     U_Output(:, model_index) = inv(A) * B;
65 end
66 %% struct
67 U_Prandalt(j).Re_tau = Re_tau;
68 U_Prandalt(j).U_damped = U_Output(:, 1);
69 U_Prandalt(j).U_undamped = U_Output(:, 2);
70 end
71 end

```

Listing 12: Prandtl Solver